

VirtualSTEMExplorerTK

Lothar Houben, Weizmann Institute of Science, October 2026

VirtualSTEMExplorerCTK is a standalone Python/Tkinter desktop application designed for real-time, interactive visual exploration and measurement of 4D-STEM (4D Scanning Transmission Electron Microscopy) datasets.

It integrates high-resolution virtual image reconstruction, flexible detector masking (annular, center-of-mass, fluctuation), real- and reciprocal-space metric rulers, calibration database synchronization, and publication-ready figure export.

A. Installation & Prerequisites

System Requirements & Dependencies

VirtualSTEMExplorerCTK requires Python 3.9+ and the following scientific packages:

`nbed`: Container library exposing `pyNBED` (HDF5 DECTRIS loader) and `MicroscopeCalibrationManager`.

I recommend installing miniconda and creating a separate Python environment for nbed. You can download and install Miniconda by following the official Anaconda Installation Guide.

Once you have installed miniconda, open a miniconda or anaconda terminal and create and activate the nbed environment:

conda create -name nbed
conda activate nbed

You may have to install python:

conda install python

Install nbed from GitHub (Note: On windows you may need to install the git utilities: Download the official installer from the Git for Windows Website):

python -m pip install git+https://github.com/LotharHouben/nbed.git@main

Download *VirtualSTEMExplorerTK.py* from https://github.com/LotharHouben/nbed/tools and save it in an executable path of your choice. Download the Microscope calibration database file *MicroscopeCalibrationDB.json* from https://github.com/LotharHouben/nbed/tools and save it in a path of your choice.

You can then start VirtualSTEMExplorerTK from its installation path:

python -m VirtualSTEMExplorerCTK

After starting the program goto ‘Settings’ and configure the path to the calibration data file under JSON DB Path. Save the setting.

Future program starts:

Open a miniconda or anaconda terminal and activate the nbed environment, navigate to the installation directory for VirtualSTEMExplorerCTK, and start the program with

python -m VirtualSTEMExplorerCTK

Troubleshooting

AMD Threadripper thread oversubscription issue: By default, numpy relies on OpenBLAS. Out-of-the-box performance can suffer due to severe thread over-subscription and cache layouts unique to Threadripper's multi-die architecture. Performance may suffer and the VirtualSTEMExplorerTK may remain unresponsive.

For AMD Threadripper machines: Add the following lines in the script code VirtualSTEMExplorerTK.py just after the module loading

```
import os

# Set these BEFORE importing numpy, scipy, or torch
os.environ["OPENBLAS_NUM_THREADS"] = "8"
# Experiment with 8, 16, or the number of physical cores in a single CCD
os.environ["MKL_NUM_THREADS"] = “8"
# Good practice to set if other packages rely on MKL
os.environ["OMP_NUM_THREADS"] = "8"
# Controls OpenMP fallback
```

Configuration & Preferences Management

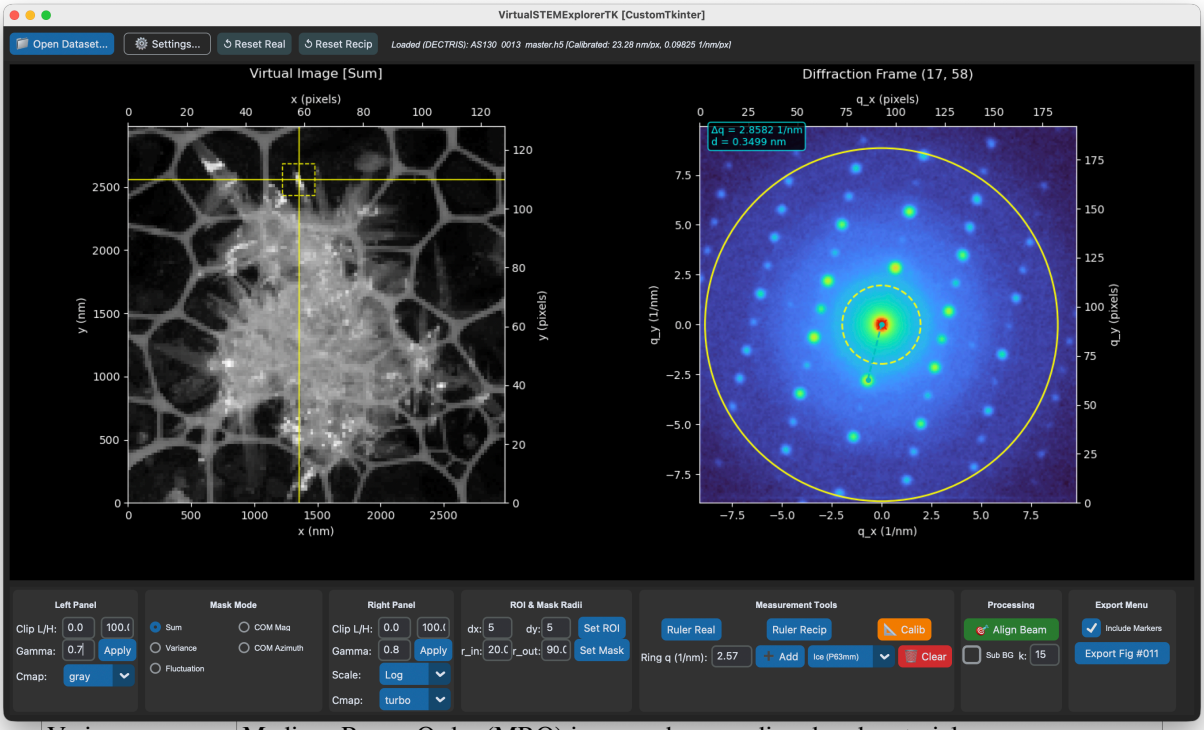
Configurations are persisted across sessions in an OS-standard location. You do not have to create the configuration file; it is auto-created upon first launch.

- **Linux/macOS:** *~/.config/VirtualSTEMExplorerCTK/VirtualSTEMExplorerCTK_config.json*
- **Windows:** *%APPDATA%\VirtualSTEMExplorerCTK\VirtualSTEMExplorerCTK_config.json*

B. Operation Manual

1. Overview & Interface Layout

VirtualSTEMExplorerCTK is an interactive CustomTkinter application designed for visual inspection, measurement, and display-level processing of 4D-STEM datasets.



Variance	Medium-Range Order (MRO) in amorphous or disordered materials
Fluctuation	Normalized Fluctuation Electron Microscopy (FEM)
COM Mag	Differential Phase Contrast (DPC) electric field mapping
COM Azimuth	Magnetic domain orientation mapping (auto-sets `hsv` colormap)

The workspace consists of three core regions:

- *Top Bar:* Quick dataset loading, preference settings, and dataset status info.
- *Dual Display Panels:* Real-space Virtual STEM reconstruction (Left) and Reciprocal-space Diffraction Frame (Right).
- *Bottom Control Dock:* Module panels for contrast adjustment, virtual mask modes, ROI settings, measurement tools, basic processing, peak finding, indexing, mapping and figure exporting.

2. Loading a Dataset

1. Click 📁 Open Dataset... on the top bar (or select File $\rightarrow$ Open Dataset...).

2. Browse and select your 4D-STEM file (.h5, .hdf5, .raw, .prz, .prz.zip).

3. Select the appropriate Data Format:

    - Auto-detect: Automatically chooses the format based on file extension.

    - DECTRIS: Standard 4D HDF5 files containing embedded scan shape metadata.

    - plain DECTRIS: Legacy DECTRIS HDF5 datasets lacking scan shape metadata. Requires manually supplying the scan shape in pixels (Y, X).

    - EMPAD: 130x128 frame raw detector binary files (.raw).

    - PantaRhei: NumPy archive structures (.prz, .prz.zip).

4. Configure Binning Settings (optional):

    - Real Space (bin_scan Y, X): Downsamples spatial scan steps upon loading to reduce memory footprint.

    - Reciprocal Space (bin_det Y, X): Downsamples detector pixel dimensions.

5. Click Load Data. Sampling calibrations (nm/px} and 1/nm/px load automatically from MicroscopeCalibrationDB.json if available, or fall back to default binning ratios.

    - *Verification:* The status bar at the top displays the loaded file name alongside the active physical pixel calibrations.

3. Real-Space ROI & Reciprocal Mask Controls

- Virtual Image Inspection: Double-Click any pixel on the Left Panel (Real Space) to update the position crosshair and instantly display the corresponding diffraction pattern on the right.

- Virtual Mask Center: Double-Click any location on the Right Panel (Reciprocal Space) to manually reposition the virtual detector center c_{qx}, c_{qy}.

- Spatial ROI Size (dx, dy): In the ROI & Mask Radii dock, enter values for dx and dy (in pixels) and click Set ROI. This averages diffraction frames over a local (2dx+1) x (2dy+1) spatial window.

    - *Verification:* The dashed yellow ROI rectangle in the Left Panel resizes to match the pixel dimensions you entered.

- Virtual Mask Radii (r_{in}, r_{out}): Enter values for (r_{in}, r_{out}) in pixels and click Set Mask to alter the virtual detector integration range.

    - *Verification:* The yellow dashed (r_{in}) and solid (r_{out}) circles on the Right Panel adjust to reflect the new integration bounds.

- Mask Mode Selection: Choose an integration algorithm in the Mask Mode dock:
    - Sum: Standard virtual bright-field or dark-field integration.
    - Variance / Fluctuation: Computes variance or relative variance across masked pixels for fluctuation electron microscopy.
    - COM Mag / COM Azimuth: Calculates Center-of-Mass momentum transfer vectors. Selecting COM Azimuth automatically switches the Left Panel colormap to hsv.

4. Adjusting Display Contrast & Scaling

Left Panel (Real Space)

- Clip L/H: Enter percentile lower/upper intensity thresholds (e.g., 0.0 to 99.5) to trim extreme outliers.
- Gamma: Enter a non-linear contrast exponent (<1.0 enhances subtle dark features; >1.0 sharpens bright regions). Click Apply.
- Cmap: Choose a real-space colormap (e.g., gray, inferno, viridis).

Right Panel (Diffraction Space)

- Scale Selector: Choose between Log (logarithmic contrast mapping) and Linear display modes via the dropdown menu.
- Clip L/H & Gamma: Independently adjust contrast clipping and gamma curves for diffraction patterns.
- Cmap: Select reciprocal-space colormaps (e.g., turbo, cividis, gray_r).
- Display Orientation: Change image display origin (lower vs upper) in ⚙️ Settings... to align visual handedness with your microscope detector setup.

5. Viewport Controls & Integration Modes

Left Viewport (Virtual STEM Reconstruction)

- Probe Selection: Click anywhere on the left panel image to move the probe position. The right panel instantly updates to display the diffraction pattern averaged over the spatial region (x±dx,y±dy).

Virtual Mask Integration Modes

Select integration modes from the Mask Mode control box:

Right Viewport (Reciprocal Detector Frame)

- Center Alignment: Click anywhere on the right panel diffraction image to re-center the virtual detector annular mask (c_{qx}, c_{qy}). The left virtual image updates immediately.
- Annular Mask Sizing: Adjust rin (inner radius) and rout (outer radius) in pixel units in the bottom dock and click Set Mask.

6. Distance & Lattice Measurements

Real-Space Measurement

1. Click Ruler Real in the Measurement Tools dock (the button highlights in red).
2. Shift-Click Point A on the Left Panel, then click Point B.
3. An overlay box displays the real-space distance Δr in nanometers (nm) and pixels (px).

- *Verification:* A magenta dashed line connects the two selected points with live coordinate readouts.

4. Click Real ON again to deactivate the measurement tool.

Reciprocal-Space Measurement & Distortion Correction

1. Click Ruler Recip in the Measurement Tools dock (the button highlights in cyan).
2. Shift-Click two reciprocal lattice spots on the Right Panel.
3. The overlay displays the magnitude of the momentum transfer Δq (1/nm) and the corresponding lattice d-spacing (d = 1/Δq in nm).
4. Reciprocal Distortion Correction:
    - Open ⚙️ **Settings...**.
    - Enter the transformation elements for the matrix

      $$M = \begin{pmatrix} \varepsilon_{11} & \varepsilon_{12} \\ \varepsilon_{21} & \varepsilon_{22} \end{pmatrix}.$$

    - Reciprocal coordinate vectors $\Delta \mathbf{q}$ are multiplied by M prior to computing distance magnitudes, correcting for elliptical distortion, projector lens anisotropy, or sample tilt.

7. Persistent Reciprocal Ring Markers

1. In the Measurement Tools dock, enter a q ring diameter (in 1/nm) into the Ring q field.

2. Click ➕ Add. A dashed cyan circle of radius $q = \text{diameter}/2$ appears centered on the direct beam location.

3. Add multiple concentric rings simultaneously (useful for matching expected powder rings, amorphous halo limits, or reference structures like hexagonal ice I_h.

4. Custom ring radii sets can be permanently configured in ⚙️ Settings... under Custom ring presets as a labeled list of (1/nm) values. They are available in a drop-down menu for quick access.

5. Click 🗑️ Clear to purge all active ring markers from the plot.

8. Centering& Background Subtraction

- 🎯 Align Beam: Click Align Beam in the Processing dock. The application tracks direct beam centroid shifts across all diffraction frames, fits a 2D linear regression surface, and applies integer pixel shifts to align the entire dataset.

- Median Background Subtraction (Sub BG):

    - Check Sub BG to subtract diffuse inelastic background scattering from the displayed diffraction frame in real time.

    - Adjust kernel size k in the input field. k is automatically constrained between a minimum of 2 and a maximum of $\frac{\det_x}{4}$ pixels.

    - *Note:* All processing operations apply strictly to display rendering and do not modify the raw underlying dataset.

9. Peak Detection

Peak detection extracts Bragg reflection positions and masses from individual or batch 4D-STEM diffraction patterns. It operates through the core function `PeakDetection()` and parallel worker chunking. Configure Parameters with live feedback, the press ‘Fit’.

PeakDetection — Parameter Settings

In `VirtualSTEMExplorerCTK`, Bragg peak detection is powered by `trackpy.locate()`, an algorithm originally developed by Crocker and Grier for particle tracking. In 4D-STEM, each diffraction disc acts like a bright "particle" on a noisy or background-subtracted detector frame. Peak detection depends on parameters; the three main parameters are

Feature Diameter, Feature Threshold, and Feature MinMass.
Below is a complete btreakdown of the parameters in `VirtualSTEMExplorerCTK`

Core Detection Parameters

- `feature_size` (`diameter`)
    - What it is: The expected physical diameter (in detector pixels) of a Bragg spot (FWTM).
    - Requirement: Must be an odd integer (e.g., 5, 7, 9, 11). If an even number is passed, the code forces `feat_sz += 1`.
    - How it works: Defines the circular window over which `trackpy` calculates the centroid and spatial moment of the intensity distribution.
    - Tuning tip: Match this to the diameter of your disc/spot. Setting it too small cuts off spot tails and skews the centroid; setting it too large blends overlapping reflections or picks up diffuse background halos.
- `feature_threshold` (`threshold`)
    - What it is: The minimum peak intensity brightness required to register as a local maximum candidate.
    - How it works: After the image is filtered (via bandpass filtering), only pixels whose processed intensity exceeds `threshold` are evaluated as potential peak centers.
    - Tuning tip: Increase this if you see spurious markers picking up noise in empty areas; decrease it to detect faint, high-angle reflections.
- `feature_minmass` (`minmass`)
    - What it is: The minimum total integrated intensity (sum of pixel values) within the feature disc.
    - How it works: Once a peak center is localized, `trackpy` sums the pixel values inside the disc of diameter `feature_size`. Peaks with an integrated mass below `minmass` are discarded.
    - Tuning tip: Unlike `threshold` (which checks peak height), `minmass` filters by total spot volume. It is effective at eliminating single-pixel hot pixels, cosmic rays, which may have high peak values but negligible integrated volume.

Spatial Separation & Pre-Filtering

- `feature_separation` (`separation`)
    - What it is: The minimum Euclidean distance (in pixels) allowed between two detected peak centers.
    - Default: Typically set to `feature_size + 1` (e.g., 8 px).
    - How it works: If two candidate peaks are closer than this distance, `trackpy` keeps only the brighter/higher-mass feature and eliminates the weaker duplicate.
    - Tuning tip: Useful for high-camera-length or convergent-beam patterns where high-order reflections or Kikuchi lines might split a single spot into multiple spurious local maxima.
- `noise_size` (`noise_size`)

    - What it is: The characteristic scale of high-frequency pixel noise (standard deviation of the Gaussian filter).
    - Default: Usually `1.0` pixel.
    - How it works: `trackpy` applies a spatial bandpass filter. The lower bound of this bandpass is controlled by `noise_size`, which suppresses pixel-level shot noise and high-frequency gain variations before peak fitting.
    - Tuning tip: Keep this around `1.0` for direct-electron detectors (e.g., EMPAD, DECTRIS). Increasing to `1.5–2.0` can help with noisy, low-dose patterns, but at the cost of slight peak broadening.
- `feature_percentile` (`percentile`)
    - What it is: Cutoff intensity percentile (from 0 to 100) below which pixels are zeroed out before candidate search.
    - Default: `0.0` (disabled).
    - How it works: If set to e.g. `64`, any pixel below the 64th percentile of the frame intensity is disregarded.
    - Tuning tip: Leave at `0.0` when using background subtraction or log scaling.
- `smooth_size` (`smoothing_size`)
    - What it is: Size of a boxcar smoothing filter applied prior to finding local maxima.
    - Default: `0` (None).
    - How it works: When greater than 0, performs local spatial smoothing. Generally left off (`None`) for STEM diffraction because Gaussian bandpassing (`noise_size`) is sufficient.

PeakDetection — Physical Calibration & Intensity Linearization

- Beam Centering: Identifies the direct beam either via the global calibrated center (`cqx`, `cqy`) or per-frame maximum intensity tracking within a search radius (typically 25 px)
- Linear Intensity Recovery: For peaks detected in logarithmic space, integrated masses are linearized using the spot area
- Distortion Correction: Peak coordinate vectors q are corrected by the distortion matrix stored in the Settings.
- Reciprocal Scaling: Physical reciprocal space radii q are scaled via `rec_samp` which requires a proper calibration.

10. Mapping

Mapping functions iterate across spatial probe positions

(i,j) in the real-space raster scan, distributing frame chunks to worker processes.

Workflow Peak Detection

Runs `PeakDetection` on every scan pixel within the designated subregion ROI or full scan grid.

- **Purpose:** Feeds non-destructive downstream reconstructions (powder histograms, selective dark-field channels, orientation maps).

Downstream Selective Reconstruction Modules

1. Powder Plot Explorer

- Radial Histogram: Accumulates radial reciprocal distances q
  q into an azimuthal profile.
- Dual-Axis Scaling: Maps reciprocal distance q and real-space distance
  Interactive Filtering:
  - Cut-Off Slider: Eliminates faint background peaks by intensity percentile.
  - SpanSelector: Allows mouse-drag bounding of specific reflection shells
    2. Selective In-Plane Orientation Explorer
    (`SelectiveOrientationMapWindow`)
- Reflection Isolation: Filters reflections in `framepeaklist` satisfying
  $q_{min} \le q \le q_{max}$.

2. In-plane Orientation Map

- Reconstructed Maps:
  - Band Intensity: Real-space dark-field map of all reflections within the selected interval.
  - Dominant In-Plane Rotation (θ): Azimuthal vector angle θ
    $\theta = \arctan2(y_{corr}, x_{corr})$ of the strongest peak in the shell.
  - Weighted Composite RGB: Modulates periodic hue (from cyclic colormaps such as `twilight` or `cet_colorwheel`) by linear or logarithmic band intensity.
- Symmetry Multiplicity (n-fold): Folds angle θ across sectors,
  $\theta_{fold} = (\theta - \theta_{Larmor}) \pmod{n 360^\circ}$

- Larmor Offset: Compensates for physical rotation between STEM scan deflector coils and the diffraction detector camera.

Workflow Real-Space 2D Lattice Strain Mapping

Computes simulated reference reciprocal vectors from an indexed zone-axis crystal template rotated by offset Δ. For each probe frame, experimental spots gexp are paired with reference reflections within a radial tolerance (`strain_tolerance`) and solves for the 2D coordinate transformation matrix A via least squaresInverts the transformation to yield real-space deformation tensor T Derived Tensors elements εxx=T11-1, , εyy =T22 −1, shear εxy =1/2 (T12 +T21), rigid rotation ω=21 (T12 −T21

Workflow Automated Crystal Orientation Mapping Workflow:

Extracts experimental spot patterns per probe position. Executes template matching against precomputed kinematical orientation plans using `py4DSTEM.process.diffraction.Crystal.match_single_pattern`. The Output are 2D Maps: Q-Map: Cross-correlation matching index (0.0≤Q≤1.0), In-plane Rotation Map (Azimuthal crystal rotation angle in degrees), Zone Axis ID Map: Symmetry-reduced zone axis identifier [uvw], R-Matrix Stack: Full 3×3 3D rotation tensor per scan pixel.

11. Exporting Figures

1. Position the real-space crosshair, virtual detector mask, measurement rulers, and contrast scaling as desired.
2. Click Export Fig #XXX in the Export Menu dock.
3. A vector-quality PDF (VirtualSTEMExplorer_Export_XXX.pdf) is saved directly into the active dataset folder, complete with calibrated physical axis ticks, secondary pixel scales, and overlay graphics.
4. The export counter increments automatically for subsequent figure exports.

Appendix

Programmatic Launch

In Python:

```
from VirtualSTEMExplorerCTK import VirtualSTEMExplorerCTK

# Launch empty GUI or load an initial file
VirtualSTEMExplorerCTK()
```

Python API State Synchronization (`target_dict`)

All user actions inside the GUI continuously update the dictionary passed to `target_dict`:

Python

```
state_dict = {}
VirtualSTEMExplorerCTK(myset, target_dict=state_dict)

# Access interactive parameters programmatically after GUI operations:
current_x = state_dict['x']
current_y = state_dict['y']
measured_d_spacing = state_dict['ruler_recip_d']
active_mode = state_dict['mode']
```

Microscope Calibration Database (MicroscopeCalibrationDB.json) Specification

Version: 2.0

File Format: JSON

Purpose: Stores microscope calibration tables, empirical field-of-view (FOV) values, and regression parameters for 4D-STEM real-space and reciprocal-space sampling calculations.

Structure Overview

The database uses a hierarchical structure where calibration datasets are indexed by **Microscope Model**, **Detector Name**, **Operating Mode**, and **Accelerating Voltage**.

```
Root
 ├── version (float)
 └── instrumenttables (object)
      └── <Instrument_Name> (object)
           └── <Detector_Name> (array of objects)
                ├── mode (string)
                ├── submode (string)
                ├── subsetting (string, optional)
                └── voltage (object)
                     └── <Voltage_in_Volts> (object)
                          ├── Magnification (object)
```

```
└── Cameralength (object)
    ├── unit (string)
    ├── interpolation (string)
    └── values (object)
```

Field Specifications

Root Level

Field	Type	Required	Description
`version`	`float`	Yes	Schema version identifier (e.g., `2.0`).
`instrumenttables`	`object`	Yes	Container holding all registered electron microscope configurations.

Instrument & Detector Hierarchy

- **`<Instrument_Name>`** (`object`): Key matching the microscope string in file metadata (e.g., `"FEI Tecnai F20"`, `"Titan Cubed 3589"`).
- **`<Detector_Name>`** (`array of objects`): Key matching the detector identifier string in metadata (e.g., `"Dectris ARINA Si"`, `"EMPAD"`). Each voltage entry in this array corresponds to a specific optical mode configuration.

Configuration Entry Object

Field	Type	Required	Description
`mode`	`string`	Yes	Illumination mode (e.g., `"STEM"`).
`submode`	`string`	Yes	Optical submode (e.g., `"Microprobe"`, `"Nanoprobe"`).
`subsetting`	`string`	No	Secondary submode or aperture configuration (e.g., `"ESI-11x7"`).
`voltage`	`object`	Yes	Map of accelerating voltages in Volts (e.g., `"200000"`, `"300000"`).

Calibration Mapping Object (`Magnification` / `Cameralength`)

Under each accelerating voltage entry, calibrations are categorized into two primary imaging types:

1. **`Magnification`**: Real-space spatial field-of-view (FOV) calibrations in nm.
2. **`Cameralength`**: Reciprocal-space field-of-view (FOV) calibrations in rec nm.

Field	Type	Required	Description
`unit`	`string`	Yes	Physical unit for the total FOV (`"nm"` for real-space, `"1/nm"` for reciprocal-space).
`interpolation`	`string`	Yes	Linear regression equation used when no exact nominal match is listed in `values`.
`values`	`object`	Yes	Key-value dictionary of empirical calibrations mapping nominal values to physical FOV values.

Calibration Lookup & Interpolation Logic

When calculating spatial sampling (pixel pitch), `MicroscopeCalibrationManager` follows a two-tier lookup algorithm:

Step 1: Nominal Settings Lookup

- **Real-Space Magnification:** The nominal magnification value M (dimensionless) is extracted from metadata.
- **Reciprocal-Space Camera Length:** The nominal camera length CL in millimeters (mm) is extracted from metadata.

Step 2: Value Resolution

1. **Exact Match:** The parser checks the `values` dictionary for a key matching the nominal setting string or float value.
2. **Regression Fallback:** If no exact match exists, the parser evaluates the `interpolation` formula.

Interpolation Formula Specification

The `interpolation` field contains a string expressing the relationship between the nominal setting x (M or CL) and the physical Field of View (FOV):

$$FOV(x) = a/x + b$$

Syntax Rule

The formula string must follow the format `"x: <expression>"`, where `x` represents the input variable:

```
text"x: 8.532E7/x + 21.207"
```

Formula Evaluation Examples

- **Real-Space FOV Calculation:**
  - **Formula:** `"x: 184700928./x + 102."`
  - **Input:** Magnification x=100,000
  - **Resulting FOV:** 1,949.01 nm
- **Reciprocal-Space FOV Calculation:**
  - **Formula:** `"x: 4137.5/x + 0.3165"`
  - **Input:** Camera length x=520.0 mm
  - **Resulting FOV:** $520.0 \cdot 4137.5 + 0.3165 = 8.273\ \text{nm}^{-1}$

Complete JSON Schema Example for MicroscopeCalibrationDB.json


```
{
  "version": 2.0,
  "instrumenttables": {
    "FEI Tecnai F20": {
      "Dectris ARINA Si": [
        {
          "mode": "STEM",
          "submode": "Microprobe",
          "voltage": {
            "200000": {
              "Cameralength": {
                "unit": "1/nm",
                "interpolation": "x: 4137.5/x + 0.3165",
                "values": {
                  "285": 15.036,
                  "520.": 8.246
                }
              },
              "Magnification": {
                "unit": "nm",
                "interpolation": "x: 8.532E7/x+21.207",
                "values": {
                  "10000": 8533.197,
                  "28500": 2976.743
                }
              }
            }
          }
        }
      ]
    }
  }
}
```